

CHECK_FRAME

Overview

check_frame verifies that a 1-bit “start” is followed by a 1-bit “test” in such a way that the clock distance between start and test falls within a predefined distance range. The distance range is specified by using the minimum (min_clks) and maximum (max_clks) values.

check_frame verification component supports reentrancy or overlapping sequences. It means that any number of “start” pulses may precede the number of “test” pulses, and the distance is checked for each pair of corresponding start and test pulses.

check_frame component allows users to associate the start and test events with a signal level, rising edge, falling edge or signal change.

In addition, **check_frame** component performs an end-of-simulation check to make sure there are no “unacknowledged” start events.

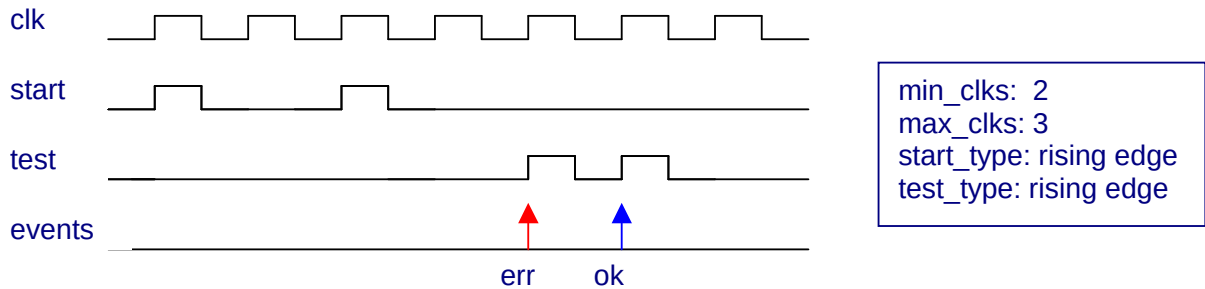
Syntax

```
check_frame #(min_clks,max_clks,start_type,test_type) InstName (clk,  
reset_n, start_event, test_event);
```

Parameters	
min_clks	Minimum distance in clock cycles between the start and test events
max_clks	Maximum distance in clock cycles between the start and test events
start_type, test_type	Enumerated types select transition types for the start and test events: 0 – signal level 1 – rising edge 2 – falling edge 3 – any change
Interface	
Clk	Clock input
reset_n	Active-low reset
start_event	1-bit signal defining a start event
test_event	1-bit signal defining a test event
Access Registers	
Distance	Monitors the actual distance in clock cycles between the start and test events. Can be used for statistics monitoring with check_stat component.
Access Events	
err, ok	Signals about failure or success.
Defines	
TOP_FINISH	Monitors the “top.finish” event, which signals to all verification components about the end of simulation. In case there is an “unacknowledged” start event(s), it issues an error to using “err” access event as well as an error display message.
ASSERT_FATAL	In case of an assertion failure, it issues a call to the top-level “top.error” task, which performs a test case exit under error conditions. Note: In order to use defines, the following rules apply: 1. Name of the top-level Testbench module is “top” 2. Top level contains an event called “finish”. This event has to fire every time before a simulation exit. 3. Top level contains a task called “error”. This task performs the exit-under-error procedure for the entire TestBench.

Usage

As it has been pointed out earlier, **check_frame** supports any number of overlapped sequences. Using built-in variable-size queue, **check_frame** applies timing checks between overlapping signals. The following figure demonstrates the way this process works:



check_distance #(2,3,1,1) demo (clk, rst_n, start, test)

Verilog Example

```
`define TOP_FINISH 1
module top;

event finish;
reg clk, rst_n;
reg req, ack;
initial clk = 1;
always #5 clk <= ~clk;

task finish_err;
begin
// this top-level task is invoked upon failure of a pre-defined check.
// Must implement an error exit procedure;
end
endtask

// Distance between each req and matched ack ranges from 3 to 6 clock cycles;
// Check is performed using rising edges of both signals

check_frame #(3,6,0,0) d1 (clk, rst_n, req, ack);

initial begin
    req = 0; ack = 0; rst_n = 0;
    #20 rst_n = 0;
    #10 req = 1;    #10 req = 0;
    #10 req = 1;    #10 req = 0;
    #10 ack = 1;    #10 ack = 0;
    #10 ack = 1;    #10 ack = 0;

    // At the end of the simulation, check that there are no non-exercised
    remainders // (non-acknowledged requests)
    #100 -> finish;
    #1 $finish;
end
endmodule
```